SHUL_V_1.7

Autonomous Multi-Agent AI Platform

Roadmap • Installation • API Guide • Kali Lab Safety • Deployment

Starter MVP — not production-ready. Real security-tool execution is disabled by default.

1. What is included

SHUL_V_1.7 includes a FastAPI backend, Master orchestration workflow, ten specialist agent roles, SQLite task records, audit events, reports endpoint, dry-run security-lab validation, Dockerfile, Render configuration, roadmap, security notes, and a master prompt. The agents are deterministic demo placeholders until an LLM provider adapter is implemented.

2. Requirements

Recommended: Python 3.11 or 3.12, Git, and a modern browser. An API key is optional for the starter. Do not commit .env or API keys to GitHub.

3. Windows installation

Open PowerShell in the extracted project directory and run: python -m venv .venv .venv\Scripts\Activate.ps1 pip install -r requirements.txt copy .env.example .env uvicorn app.main:app --reload

Open http://127.0.0.1:8000/docs to explore the API.

4. Linux installation

Run: python3 -m venv .venv source .venv/bin/activate pip install -r requirements.txt cp .env.example .env uvicorn app.main:app --host 0.0.0.0 --port 8000

5. Test the API

```
GET /health
GET /api/agents
POST /api/tasks
Request body:
{"command":"Create a portfolio website plan","mode":"software"}
GET /api/reports/1
```

The API returns task status and a structured report. Use the actual task ID returned by your request.

6. Ten specialist agents

Planner, Researcher, Architect, Coder, Tester, Reviewer, Security, Data/ML, Reporter, and Learning. In this MVP they return safe demonstration summaries. Connect real models only after adding schema validation, provider timeouts, rate limits, source citations, prompt-injection defenses, and evaluation tests.

7. Kali Linux integration

Run Kali Linux in a separate VM, preferably on a host-only or isolated lab network. Do not run the AI service as root or expose a privileged Docker socket. This package does not execute Kali tools. The security endpoint validates a configured lab target and returns a dry-run plan. Only test systems you own or have explicit written authorization to assess.

8. Online deployment

Push the project to a private GitHub repository and deploy with Render using render.yaml or Dockerfile. Add environment variables through the hosting dashboard. Before public use, add authentication, role-based access, rate limits, persistent database storage, monitoring, and a security review. Free services may sleep or have storage limitations.

9. Voice, images, and video

The current package accepts text commands through the API. Voice transcription, text-to-speech, image understanding, and video frame analysis are integration points for a later frontend. Apply upload limits, content-type checks, malware scanning, privacy controls, and consent rules before enabling uploads.

10. Autonomous learning and project creation

Use controlled improvement: collect feedback, create evaluation cases, propose reusable skills, test changes in a sandbox, run static analysis and tests, and require reviewer/Admin approval before merging or deploying. Do not let agents modify their own safety rules or production environment without review.

11. Production checklist

Add login and RBAC; secure secret storage; background task queue; per-task resource limits; isolated code execution; dependency/SBOM scanning; backups; structured logs; monitoring; provider budgets; tests; privacy retention policy; and a threat model. Treat the current scaffold as a development baseline, not a finished commercial platform.